Vietnam National University Ho Chi Minh City

University of Science

Faculty of Information Technology

Department of Software Engineering

PERSONAL DOCUMENT

Final Project Report

Course: Mobile Application Development (CS426)

Project members:

Trinh Vo Nam Kiet – 24125013

Doan Tuan Anh – 24125002

Tran Khanh Nhan – 24125037



September 2026

Contents

1 Project Overview **3**
1.1 Course Context and Application . . . 3
1.2 Problem Statement and User Pain Points . . . 3
1.3 Proposed Solution . . . 4

2 Project Scope **5**
2.1 Target Users . . . 5
2.2 Target Platform . . . 5
2.3 Project Goals . . . 6
2.4 Non-goals (Out of Scope) . . . 6
2.5 Setup and Installation . . . 7

3 System Architecture **8**
3.1 Architecture Overview . . . 8
3.1.1 Clean Architecture Rationale . . . 8
3.1.2 Dependency Injection with `AppContainer` . . . 9
3.1.3 Presentation Architecture . . . 9
3.2 Architecture-to-Project Mapping . . . 9
3.3 System Execution Flows . . . 10
3.3.1 Document Processing Flow . . . 10
3.3.2 Hybrid Search Flow . . . 11

4 Technical Implementation **12**
4.1 Functional Pipeline . . . 12
4.1.1 Online Processing Pipeline . . . 12
4.1.2 Offline Processing Pipeline . . . 13
4.1.3 Metadata Review . . . 14
4.1.4 Hybrid Search Engine . . . 15
4.2 User Interface and Navigation . . . 16

5 Future Work and Technical Roadmap **17**
5.1 On-Device Language Models . . . 17
5.2 Feature Extensions . . . 18
5.2.1 Repository Question Answering . . . 18
5.2.2 Deadline and Follow-Up Notifications . . . 18

6 Self-Assessment **19**

7 References **20**

Work Division

Student ID	**Full Name**	**Main Responsibilities**
24125013	Trinh Vo Nam Kiet	Implemented the offline document-processing workflow, including OCR-based metadata generation, object detection, and local embedding-based search.
24125002	Doan Tuan Anh	Implemented the online Gemini-based processing pipeline, OCR and detection integration, and AI-assisted document analysis.
24125037	Tran Khanh Nhan	Designed and implemented the application UI, including the main screens, navigation flow, and data models used across the app.

Table 1: Work division among project members

1 Project Overview

1.1 Course Context and Application

The **Personal Document** project is developed as the capstone final project for the course **Mobile Application Development (CS426)** at the University of Science, Vietnam National University Ho Chi Minh City (HCMUS).

Personal Document is a native Android application (supporting Android 7.0, API Level 24 and higher, built entirely with Kotlin, Jetpack Compose, Material Design 3, Room, MediaPipe, ML Kit, and Coroutines/Flow). The application functions as an intelligent personal digital repository, empowering users to digitize, automatically analyze, structure, and search through photo-captured documents, receipts, study notes, and personal credentials.

1.2 Problem Statement and User Pain Points

In our fast-paced daily study and professional lives, the smartphone camera has naturally evolved into our primary instantaneous memory capture tool:

- **Students** snap photos of lecture slides, whiteboard derivations, textbook excerpts, and handwritten peer study notes.
- **Working professionals and freelancers** photograph dining receipts, fuel bills, travel invoices for expense reimbursement, client contracts, and business cards.
- **Everyday individuals** capture citizen IDs, health insurance cards, medical prescriptions, home appliance warranties, and utility meter readings.

The Everyday User Dilemma: Imagine standing at a government administrative office or facing the monthly company expense audit deadline. You distinctly recall photographing a business lunch receipt or an insurance policy card three weeks ago. You open your default system Gallery app. Immediately, you are confronted with a disorganized labyrinth of thousands of photos: holiday snapshots, food photos, selfies, downloaded memes, social media screenshots, and family videos.

The filenames offer no assistance, represented merely by arbitrary computer-generated timestamps such as `IMG_20260905_143022.jpg`. You find yourself scrolling endlessly, squinting through hundreds of tiny thumbnails, desperate to locate that single piece of paper. This process easily squanders 10 to 15 minutes of frustration, and in many instances, users give up entirely, forfeiting financial reimbursement or missing critical deadlines.

Core Limitations of Existing Solutions:

1. **Visual Pollution and Document Fragmentation:** Critical documents are dumped alongside casual photos in the device gallery, lacking any dedicated workspace.
2. **Primitive Search Capabilities:** Traditional mobile galleries index media solely by timestamp or rough GPS location. When a user remembers the conceptual contents (e.g., *"Phuc Long coffee invoice"* or *"B-Tree database index slide"*), system galleries remain entirely helpless.
3. **Heavy Manual Data Entry Friction:** Conventional scanning and note-taking apps require users to manually type titles, select categories, and compose tags. This tedious overhead causes cognitive friction, discouraging consistent organizational habits.
4. **Privacy Concerns and Network Dependence:** Cloud-exclusive document scanners upload sensitive personal and financial credentials to third-party servers, posing serious data privacy risks and rendering the application useless during cellular dead zones or offline travel.

1.3 Proposed Solution

Personal Document directly addresses these challenges by merging an adaptive **Cloud-First** architecture with a dependable **Offline On-Device Machine Learning** pipeline:

- **Isolated, Private Repository:** Scanned and imported documents are organized in an app-private library, completely segregated from everyday media in the Android system gallery.
- **Automated Metadata Generation Pipeline:** The moment an image is captured or selected, the system autonomously:
  - Transcribes all text via optical character recognition (OCR).
  - Identifies visible real-world objects through image labeling.
  - Generates a concise title, a single-sentence descriptive summary, predicts the appropriate category (*NOTE, RECEIPT, IDENTITY, STUDY, WORK, OTHER*), and suggests up to 5 topical tags.
- **Human-in-the-Loop Verification:** All AI-generated fields are treated strictly as suggestions encapsulated within a `DocumentDraft`. Users inspect and can freely modify any field on an interactive Review screen before confirming persistence.
- **Fuzzy Hybrid Search Engine:** The application combines semantic vector similarity (cosine similarity over 512-dimensional embeddings) with typo-tolerant lexical

matching (Levenshtein distance). Users can locate documents via conceptual queries or misspelled keywords effortlessly.

- **Cloud-First with Resilient Local Fallback:** The application defaults to Google's Gemini Vision API when online for maximum multimodal reasoning, but gracefully falls back to local Google ML Kit and MediaPipe on-device models when offline or if disabled by user settings.

2 Project Scope

2.1 Target Users

The application is specifically engineered for several distinct user demographics:

1. **Students, Researchers, and Educators:**
   - Frequently capture lecture slides, whiteboard solutions, textbook diagrams, academic papers, and homework sheets.
   - Require systematic categorization under the *STUDY* domain and rapid retrieval of formulas, definitions, or exam topics during revision.
2. **Office Workers, Freelancers, and Small Business Owners:**
   - Need to capture, track, and aggregate commercial receipts, meal vouchers, travel tickets, and invoices for monthly financial accounting and expense claims (*RECEIPT*).
   - Organize project plans, meeting agendas, physical business cards, and signed contracts under the *WORK* category.
3. **Everyday Individuals and Household Organizers:**
   - Maintain crucial personal identification credentials such as National IDs, Passports, Driver's Licenses, and Medical Insurance Cards (*IDENTITY*).
   - Individuals prone to forgetfulness who benefit from an intelligent search engine that understands conceptual context rather than requiring exact keyword memory.

2.2 Target Platform

- **Operating System:** Native Android OS, with full backward compatibility spanning from **Android 7.0 (API Level 24 – Nougat)** up to the latest releases (**Android 14 and 15**).

- **Hardware Requirements:** Designed to execute efficiently on mid-range and budget Android smartphones (requiring a minimum of 3GB to 4GB of RAM and a standard autofocus rear camera module for clear optical capture).
- **Operating Environment:** Built to perform seamlessly in dual operating conditions: high-speed network connectivity (leveraging cloud APIs) and complete offline autonomy (zero cellular signal or Wi-Fi).

2.3 Project Goals

The core engineering objectives are as follows:

- **Strict On-Device Privacy and Centralized Storage:** Images and metadata are securely quarantined within the application's private sandbox storage and local Room Database, guaranteeing zero unwanted leakage into the public Android media gallery.
- **Automated Metadata Extraction:** Eradicate manual data entry friction by automatically transcribing textual contents (OCR), generating titles, composing descriptive summaries, inferring categories, and recommending topical tags using Machine Learning.
- **Intelligent Hybrid Search Engine:** Construct a resilient search pipeline integrating dense semantic vector embeddings (MediaPipe) with typo-tolerant fuzzy lexical matching (Levenshtein distance).
- **Cloud-First Architecture with Graceful Local Fallback:** Capitalize on state-of-the-art Multimodal LLMs (Google Gemini Vision API) when online, while ensuring continuous 100% operational readiness through local on-device ML models (ML Kit, MediaPipe) whenever network connectivity degrades or fails.
- **Human-in-the-Loop User Experience:** Empower users to inspect, refine, or override AI suggestions on an interactive Review screen prior to permanent storage.

2.4 Non-goals (Out of Scope)

To maintain a sharply defined scope, safeguard user privacy, and deliver optimal on-device performance within the academic timeline, the following features are intentionally classified as non-goals:

- **No Multi-Device Cloud Synchronization:** The system does not maintain an external remote server backend for syncing images across different user devices. All data remains exclusively on the client device.

- **No Account Creation or User Authentication:** Adhering to a friction-free philosophy, the app requires no sign-up, login credentials, or email verification; users open and utilize the application immediately.
- **No Collaborative or Shared Access:** The app is purpose-built for single-user personal document management, excluding multi-tenant sharing, document permissions, or live collaborative annotations.

2.5 Setup and Installation

The application is designed for straightforward setup on a standard Android development machine. The following steps are required for successful local building and execution:

1. **Install prerequisites:** Download and install Android Studio, the latest Android SDK, and the Android Build Tools recommended by the project. Ensure that the Java Development Kit (JDK) is configured correctly and that the Android Gradle Plugin can sync successfully.
2. **Clone the project:** Obtain the source code from the repository and open the root folder in Android Studio.
3. **Create `local.properties`:** In the project root, create a file named `local.properties` containing the path to the Android SDK. If the application is configured to use the Gemini Vision API, the file may also include the API key as follows:

```
sdk.dir=C\:/Users/<YourUser>/AppData/Local/Android/Sdk
GEMINI_API_KEY=your_gemini_api_key_here

```

4. **Synchronize Gradle:** Let Android Studio finish dependency resolution and Gradle sync. If a dependency or SDK package is missing, allow the IDE to install the required components.
5. **Run the app:** Connect an Android device via USB or start an emulator, then click `Run > Run 'app'` in Android Studio. The application will build the debug variant and launch it on the selected device.
6. **Command-line build:** The project can also be built from a terminal using the Gradle wrapper:

```
./gradlew assembleDebug

```

   On Windows, use:

```
    .\gradlew.bat assembleDebug

```

7. **Offline fallback behavior:** If the `GEMINI_API_KEY` variable is absent or the network call fails, the application automatically falls back to the on-device ML pipeline using ML Kit and MediaPipe. This ensures the app remains functional even when external connectivity is unavailable.

3 System Architecture

3.1 Architecture Overview

The application architecture is structured on three foundational engineering pillars: Robert C. Martin's **Clean Architecture**, **Manual Dependency Injection (DI)** via an `AppContainer` composition root, and the **Model-View-ViewModel (MVVM)** pattern coupled with **Unidirectional Data Flow (UDF)** in the presentation layer.

3.1.1 Clean Architecture Rationale

In mobile applications integrating rapidly advancing Artificial Intelligence and Machine Learning (AI/ML) capabilities, system requirements and external dependencies shift frequently: a cloud API endpoint may change, a local neural network model may be upgraded, or a new indexing strategy may be tested. Adopting Clean Architecture guarantees:

1. **Complete Independence from External Frameworks:** The core business layer (**Domain Layer**) is implemented in pure Kotlin. Use cases such as `AnalyzeDocument`, `SearchDocuments`, and `SaveDocument` maintain zero dependencies on Android SDK classes or specific ML libraries. They interact exclusively through abstract interfaces (`DocumentRepository`, `OcrService`, `EmbeddingService`, `LlmService`).

2. **Pluggable and Interchangeable Service Implementations:** Transitioning from a cloud-based multimodal model (Gemini Vision API) to an on-device pipeline (ML Kit OCR + MediaPipe Summarizer) requires merely swapping the concrete implementation in the composition root without altering a single line of business or UI code.

3. **High Automated Testability:** Business rules, search ranking algorithms, and similarity metrics can be validated via fast JVM unit tests using lightweight test doubles (e.g., `FakeDocumentDao`, `FakeEmbeddingService`) without requiring an Android emulator or physical hardware.

4. **Strict Separation of Concerns:** Presentation, Domain, and Data logic remain strictly isolated, simplifying collaborative git branching and preventing spaghetti dependencies.

3.1.2 Dependency Injection with `AppContainer`

Rather than relying on heavy compile-time annotation processors such as Dagger-Hilt (which introduce significant build latency and verbose generated code), the project utilizes a clean **Manual Dependency Injection** pattern anchored by `AppContainer`:

- `AppContainer` instantiates and manages the lifecycles of singleton dependencies: the local Room Database (`AppDatabase`), the image storage service, ML services (ML Kit, MediaPipe), and the Gemini API client.
- It wires concrete dependencies into Domain Use Cases (`ObserveDocuments`, `SearchDocuments`, `SaveDocument`, etc.).
- It exposes customized `ViewModelProvider.Factory` instances, ensuring that ViewModels receive only the precise use cases necessary for their execution.

3.1.3 Presentation Architecture

- **View (Composables):** Built declaratively using Jetpack Compose. Views observe an immutable state (`UiState`) emitted by the ViewModel and propagate user interactions as explicit actions/events.
- **ViewModel:** Extends `androidx.lifecycle.ViewModel`, hosts a backing `MutableStateFlow<Ui`, executes asynchronous workflows via Kotlin Coroutines, and updates the observable UI state.
- **Unidirectional Data Flow (UDF):** State flows downward from ViewModel to Composable screens; user events flow strictly upward from Composables into ViewModel entry points, preventing UI race conditions.

3.2 Architecture-to-Project Mapping

The three concentric Clean Architecture tiers map directly to the codebase package structure:

Layer	Core Components	Project Location

Presentation	Compose Screens, ViewModels, UI State definitions, Navigation	`presentation/capture/`
		`presentation/review/`
		`presentation/library/`
		`presentation/detail/`
		`presentation/setting/`
		`navigation/AppNavGraph.kt, Routes.kt`

Domain	Entities, Domain Models, Use Cases, Service Contracts	`domain/model/`
		`domain/usecase/`
		`domain/service/`
		`domain/repository/`

Data	Room Database, DAO, Entity Mappings, ML Adapters	`data/local/`
		`data/repository/`
		`data/ml/`

Table 2: Detailed architectural component and package directory correlation

3.3 System Execution Flows

The following sequence diagrams illustrate the runtime coordination among layers during core user journeys (extracted from `docs/DIAGRAMS.md`):

3.3.1 Document Processing Flow

The sequence below demonstrates image capture, automatic metadata generation (Gemini Vision or On-Device fallback), presentation of the interactive Review screen, and embedding computation upon user confirmation:

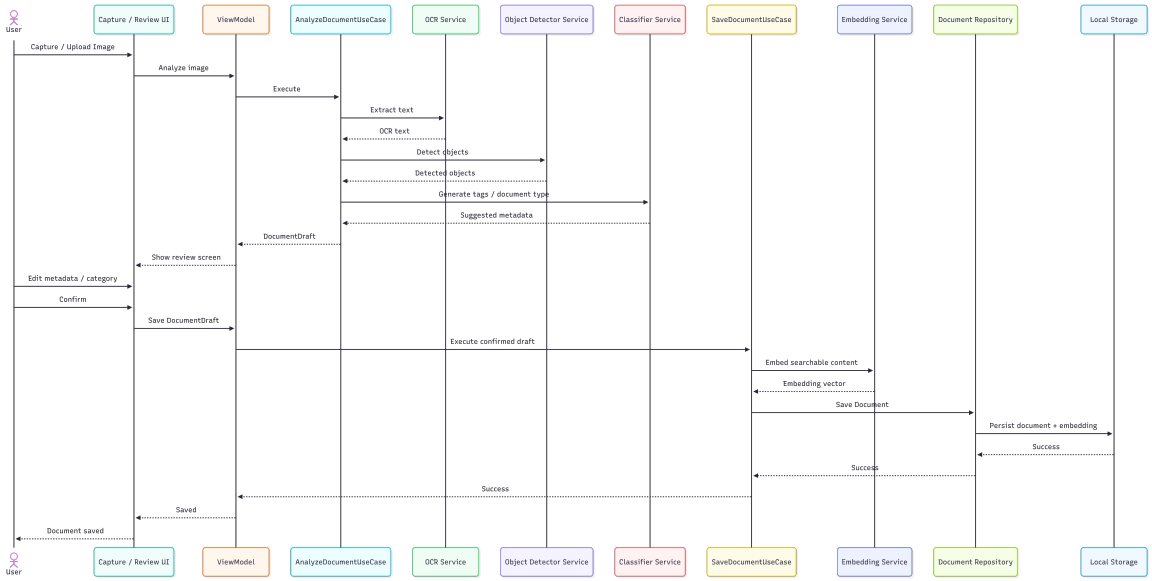


Figure 1: Document analysis flow

3.3.2 Hybrid Search Flow

The sequence below traces a real-time search query from user keystroke debouncing to embedding generation, similarity ranking, and reactive UI updates:

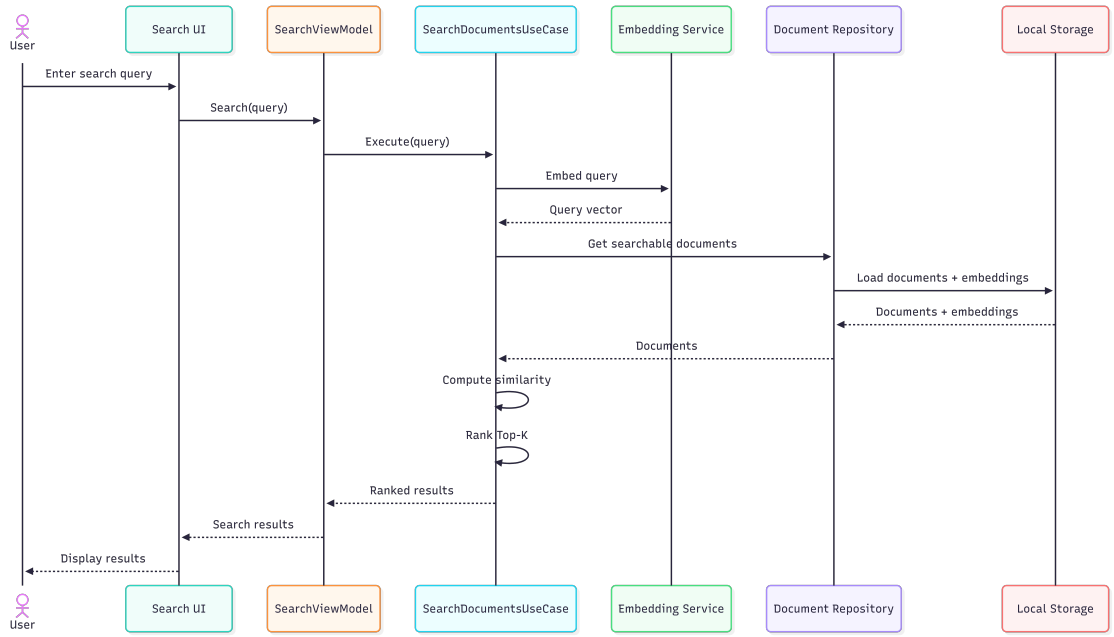


Figure 2: Hybrid search flow

4 Technical Implementation

4.1 Functional Pipeline

The core operational lifecycle of the system operates in a continuous, end-to-end loop:

Input Image ⟶ Extract Metadata ⟶ Vector Embedding ⟶ Retrieval and Ranking

To balance high-accuracy reasoning with offline resilience, the application follows a **Cloud-First with On-Device Fallback** paradigm comprising two interchangeable execution branches:

4.1.1 Online Processing Pipeline

When the device possesses active internet connectivity and an authorized API key is configured (`BuildConfig.GEMINI_API_KEY`), the application prioritizes Google's **Gemini 3.1 Flash Lite** (or **Gemini 3.1 Flash**) multimodal model managed by `GeminiApiLlmService`.

Execution Process: The raw image is read from local storage into a byte array (`ByteArray`) and dispatched alongside a structured prompt instructing the model to act as an advanced document analysis assistant.

The LLM is constrained to respond with **ONLY a valid JSON object** (excluding code fences or narrative commentary) adhering strictly to the typed schema:

```
{
  "title": "Short human-readable title (max 60 characters)",
  "description": "One concise sentence summarising what the document
   contains",
  "ocrText": "Full plain text read or extracted from the document image
   ",
  "category": "NOTE | RECEIPT | IDENTITY | STUDY | WORK | OTHER",
  "tags": [
    "keyword1",
    "keyword2",
    "keyword3",
    "keyword4",
    "keyword5"
  ],
  "detectedObjects": [
    "paper",
    "document",
    "receipt"
  ]
}
```

Listing 1: Structured JSON response schema returned by Gemini Vision API

Core Advantages: Leveraging large multimodal parameters, Gemini effortlessly deciphers skewed text orientations, low-contrast scans, handwritten notes, and irregular receipt layouts, yielding top-tier semantic categorization and contextual summaries.

4.1.2 Offline Processing Pipeline

Whenever network calls time out, encounter HTTP quota limits, or when the user explicitly toggles the **"Disable API"** setting in `SettingScreen`, the system seamlessly activates the local on-device ML suite:

4.1.1. OCR Service (Google ML Kit Text Recognition):

- Implemented in `MlKitOcrService`.
- Uses Google ML Kit's Latin script text recognizer operating entirely on-device (leveraging NNAPI/GPU hardware acceleration when available).
- Extracts bounding text blocks, lines, and tokens from the bitmap without transmitting any data over the network.

4.1.2. Image Labeling / Object Detection (Google ML Kit):

- Implemented in `MlKitImageLabelingService`.
- Utilizes `ImageLabeling.getClient` configured with a confidence threshold ≥ 0.70.
- Detects prominent physical items (e.g., *Paper, Document, Receipt, Book, Invoice, Signature*) to support semantic classification.

4.1.3. Description Generation Service (Google MediaPipe Tasks):

- Implemented in `MediaPipeDescriptionGenerationService`.
- Employs the **MediaPipe Text Summarizer** model executing locally in `TLDR` mode (`MAX_NUM_TOKENS = 64`).
- The model binary is pre-bundled within `assets/` and copied to the internal application cache for instantaneous inference.
- Formulates a coherent single-sentence overview summarizing the OCR text and detected objects.

4.1.4. Tag Generation Service:

- Implemented in `SimpleTagGenerationService`.
- Fuses object labels with salient tokens filtered from the OCR transcription.

- **Bilingual Stop-Word Filtering:** Strips irrelevant tokens across English (*the, and, for, with...*) and Vietnamese (*cua, va, la, cac, trong, cho, ngay, thang, nam...*).
- **Domain Keyword Prioritization:** Elevates high-value domain tokens (*receipt, invoice, passport, identity, license, certificate, contract, lecture, homework, report, project*).
- Computes word occurrence frequencies to select the top 5 most representative tags.

4.1.5. Category Classification Service (Semantic Category Matching):

- Implemented in `EmbeddingCategoryClassificationService`.
- Defines dense semantic anchor descriptions for standard target categories:
  - `NOTE`: *"personal note, memo or reminder"*
  - `RECEIPT`: *"receipt, invoice, purchase, payment, subtotal, or total"*
  - `IDENTITY`: *"passport, id card, driving license, official identification, certificate"*
  - `STUDY`: *"lecture notes, textbook, exam, homework, course, syllabus, student"*
  - `WORK`: *"meeting notes, project plan, contract, business report, presentation, employment"*
- Embeds these category definitions into 512-dimensional vector space using `MediaPipeEmbeddingService`.
- During document classification, the aggregated document text is vectorized and matched against category embeddings using **Cosine Similarity**. The category yielding the highest score above 0.30 is assigned; otherwise, the document defaults to `OTHER`.

4.1.3 Metadata Review

A crucial architectural decision is that **no raw AI output is saved directly into the database**:

- Output fields from Gemini or the local ML pipeline are bundled into an immutable `DocumentDraft`.
- The user is automatically transitioned to `ReviewScreen`, where they are presented with the captured image alongside the pre-filled metadata.
- Users maintain complete authority to edit titles, adjust summaries, correct misclassified categories, or add/delete tags via interactive Material 3 chips.

- **Final Persistence:** Only when the user taps **"Confirm/Save"** is the finalized metadata synthesized into a searchable document string:

$$\text{searchableText} = \text{title} + \text{description} + \text{ocrText} + \text{tags} + \text{detectedObjects}$$

This consolidated string is vectorized by `MediaPipeEmbeddingService` and stored alongside the document record in Room Database.

4.1.4 Hybrid Search Engine

Search queries initiated on `LibraryScreen` utilize a hybrid ranking formulation balancing conceptual understanding with textual precision:

$$\text{TotalScore} = 0.8 \times \text{SemanticScore} + 0.2 \times \text{LexicalScore} \tag{1}$$

1. Semantic Vector Score: Determined by cosine similarity between the query embedding vector and the stored document vector, which measures the degree of alignment between their vector directions.

2. Fuzzy Lexical Score: To accommodate spelling typos (e.g., searching *"databse"* instead of *"database"*), the system computes a normalized similarity based on the Levenshtein edit distance:

$$\text{fuzzySimilarity}(a, b) = \begin{cases} 1.0 & \text{if } \max(|a|, |b|) = 0 \\ 1.0 - \dfrac{\text{levenshteinDistance}(a, b)}{\max(|a|, |b|)} & \text{otherwise} \end{cases} \tag{2}$$

Given query terms Q and document terms D, the aggregate lexical score represents the mean of each query term's maximum similarity:

$$\text{LexicalScore} = \frac{1}{|Q|} \sum_{q \in Q} \left(\max_{d \in D} \text{fuzzySimilarity}(q, d) \right) \tag{3}$$

3. Dual-Threshold Result Filtering: Documents are retained in the search results if and only if they satisfy the dual-threshold condition:

$$\text{Retain} \iff (\text{TotalScore} \geq 0.25) \ \vee \ (\text{LexicalScore} \geq 0.50) \tag{4}$$

This rule ensures that strong semantic matches are surfaced even when exact wording differs, while near-identical lexical matches (e.g., misspelled keywords with LexicalScore ≥ 0.50) are preserved even if semantic embeddings yield a modest score.

4.2 User Interface and Navigation

The application comprises five cohesive screens orchestrated through `AppNavGraph`:

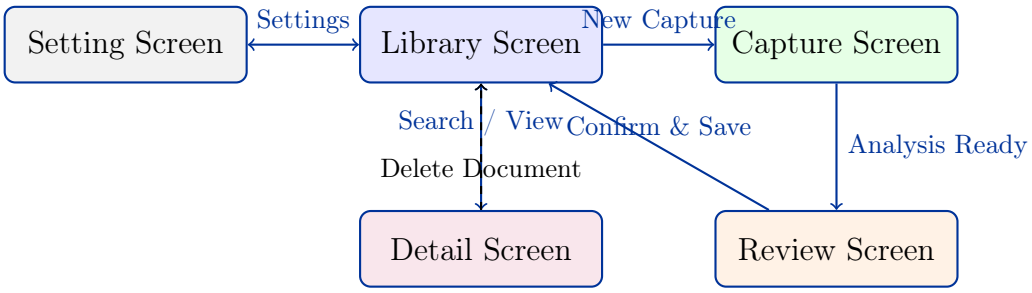


Figure 3: Screen navigation state graph (App Navigation Flow)

1. **LibraryScreen (Primary Repository & Live Search):**
   - Entry point displaying stored items in a responsive grid/list view (`DocumentCard`).
   - Top search bar featuring a 100ms debounce delay for fluent, stutter-free real-time queries.
   - Displays category badges, topical tags, creation dates, and debug score badges (`sem: xxx`, `lex: xxx`).
2. **CaptureScreen (Document Capture & Media Import):**
   - Integrates CameraX for fast auto-focus and tap-to-focus document imaging.
   - Supports importing pre-existing document images from device storage via the Android Photo Picker.
3. **ReviewScreen (Human-in-the-Loop Inspection & Editing):**
   - Renders captured imagery alongside AI-inferred suggestions.
   - Displays a prominent fallback warning banner if the cloud API failed and on-device fallback was used.
   - Provides editable text fields and category selector chips for user adjustments before final database commit.
4. **DetailScreen (Full Document Inspection & Zoom Viewer):**
   - Presents complete metadata, transcribed OCR text blocks, and timestamps.
   - Integrates `FullScreenImageViewer` with two-finger pinch-to-zoom and pan gestures.
   - Facilitates document editing or deletion with atomic storage cleanup.

5. **SettingScreen (System Configuration & Diagnostics):**
    - **API Toggle:** Allows users to disable external API calls to enforce complete offline privacy.
    - **Camera Start Mode:** Option to launch directly into the capture view upon app startup.
    - **Factory Reset:** Wipes Room Database records, DataStore preferences, and internal image caches to restore initial state.

5 Future Work and Technical Roadmap

5.1 On-Device Language Models

The current **Cloud-First with Local Fallback** architecture delivers a dependable balance between multimodal precision and continuous offline utility. Nevertheless, empirical usage highlights distinct limitations when operating strictly offline:

- **Heuristic OCR Limitations on Unstructured Media:** Traditional mobile OCR engines (such as ML Kit Text Recognition) perform exceptionally well on high-contrast printed typography, but experience noticeable degradation when parsing freehand handwriting, skewed receipt margins, faint inks, or intricate tabular matrices.
- **Cascading Metadata Degradation:** Because downstream tasks (title generation, text summarization, category classification, and keyword tagging) rely upon OCR tokens, noisy optical transcription directly degrades downstream accuracy, requiring heavier user manual corrections on the Review screen.

Prospective Architectural Evolution: Directly embedding quantized **Small Language Models (SLMs)** within the local mobile runtime via Google MediaPipe GenAI SDK, ONNX Runtime Mobile, or ExecuTorch. Candidate models include:

- **Google Gemma 2B / Gemma-4N:** Tailored for mobile acceleration on modern NPU and GPU architectures.
- **Qwen 2.5 (0.5B / 1.5B):** Exceptionally compact footprint, rapid token generation, and robust multilingual comprehension (particularly English and Vietnamese).
- **LLaMA-3.2 1B/3B Mobile:** 4-bit integer quantization (*INT4*) enabling execution within mobile thermal envelopes.

Hardware Constraints and Pragmatic Trade-offs: While on-device SLMs are technically compelling, mobile devices in entry-level segments with limited RAM ($\leq$ 4GB) risk severe Out-Of-Memory (OOM) exceptions and thermal throttling when allocating 1.5GB to 3GB model weights into working memory. Consequently, the optimal architectural roadmap involves a **Dynamic Hardware Capability Detector**: devices equipped with ample RAM ($\geq$ 8GB) and dedicated NPUs will execute local SLMs, while standard devices will continue leveraging the lightweight ML Kit and MediaPipe Embedder pipeline.

5.2 Feature Extensions

5.2.1 Repository Question Answering

A natural extension of semantic search is upgrading from ranked retrieval to full **Retrieval-Augmented Generation (RAG)** across the user's personal document collection (analogous to an on-device personal NotebookLM):

- Enabling conversational queries over multi-document contexts:
  - *"What was my total expenditure on client lunches last month?"*
  - *"When does my air conditioner warranty expire?"*
  - *"Summarize all key formulas from the database indexing lecture."*
- **Feasibility Assessment:** While technically promising, multi-document synthesis requires significant vector ranking pipelines and context management. In alignment with the project's primary objective—providing instantaneous retrieval for forgetful users with zero lag—this capability is prioritized for subsequent major revisions.

5.2.2 Deadline and Follow-Up Notifications

Physical documents frequently involve critical temporal triggers:

- Utility bills and credit card statements requiring settlement by a specific calendar date.
- Saved articles, technical tutorials, or reading materials captured for weekend review.
- Medical follow-up appointments, passport renewal milestones, or flight departures.

Proposed Implementation: Integrating an on-device **Temporal Entity Extractor** within the metadata pipeline. Upon detecting payment deadlines or appointment dates:

- The system automatically prompts the user to schedule a local reminder.
- Leverages Android's `AlarmManager` and `WorkManager` to dispatch reliable local notifications (*Local Push Notifications*) without needing external push notification servers.

6 Self-Assessment

This project was completed as a collaborative Android application development effort aimed at solving a realistic personal-document management problem. Overall, we believe the system successfully meets the core requirements of the course: it is implemented as a native Android application with multiple connected screens, persistent local storage, and integrated AI/ML-based document-processing features. The app supports image capture or import, automatic OCR and metadata extraction, classification, search, and review before persistence.

What we completed successfully:

- The app provides a dedicated personal document library separated from the system gallery, allowing safe and organized storage of scanned or captured documents.
- The pipeline for document analysis works in both online and offline modes. The online mode leverages Gemini for richer multimodal analysis, while the offline mode relies on local ML Kit and MediaPipe components to ensure functional availability without internet access.
- The application includes a hybrid search system that combines semantic embedding similarity with fuzzy lexical matching, improving recall when the user remembers only approximate phrases or misspelled keywords.
- The UI was designed to support a clear user flow from capture, to review, to storage, and then to search or detail inspection.
- The project follows a clean separation between presentation, domain logic, and data infrastructure, which makes the codebase more maintainable and easier to extend.

What we improved and learned:

- We strengthened our understanding of Android architecture patterns, especially MVVM, Flow-based state handling, Room persistence, and dependency injection through a custom container.
- We learned how to integrate AI/ML capabilities into mobile workflows while balancing accuracy, efficiency, and privacy requirements.
- We improved our ability to design an app around real user pain points, focusing not only on visual interface quality but also on practical retrieval and organization workflows.

Current limitations:

- The system is designed as a single-user personal repository rather than a collaborative or multi-device platform.
- In fully offline conditions, document accuracy can still depend on OCR quality and local model limitations, especially for handwritten or highly distorted content.
- The system currently focuses on local document management and retrieval rather than full multi-document reasoning or advanced RAG-style question answering.

Overall assessment: We consider the project to be a successful capstone implementation for the course. It demonstrates both technical depth and practical value, combining Android development, local persistence, AI/ML processing, and user-centered design in a working application. The project not only meets the course's functional expectations but also provides a solid foundation for future extensions such as richer document understanding, reminders, and expanded personal knowledge retrieval.

7 References

- Martin, Robert C. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- Google Developers. *Guide to app architecture*. Android Developers Documentation. Available at: https://developer.android.com/topic/architecture
- Google Developers. *Jetpack Compose documentation*. Android Developers Documentation. Available at: https://developer.android.com/jetpack/compose
- Android Developers. *Android documentation*. Available at: https://developer.android.com/
- Google ML Kit. *ML Kit documentation*. Available at: https://developers.google.com/ml-kit
- Google MediaPipe. *MediaPipe documentation*. Available at: https://developers.google.com/mediapipe
- Google AI for Developers. *Gemini API documentation*. Available at: https://ai.google.dev/gemini-api/docs